Experiment-1

Demonstrate and Create project in local and remote repository using GitBash and GitHub and apply init, status, log, add, commit, push, config, clone and reset commands on repository.

Software Required

- A computer with Windows
- **Git Bash** installed → https://git-scm.com/
- **GitHub account**
- Internet connection

Step 1 — Create a Local Project (Either you can create it manually or through gitbash

mkdir git-lab-project
cd git-lab-project

or

Create the new folder on desktop manually

Step 2 — Initialize Local Repository

Go to the new folder you have created > right click > open the gitbash and give the below command

git init

Creates a new **Git repository** in your project folder.

Step 3- Creating the files

Give the commands:

touch abc.txt

touch index.html

touch p.python

Step 4 — Configure Git

Set up your username and email (one-time setup):

```
git config --global user.name "Your Name"
git config --global user.email "yourmail@example.com"
```

Verify configuration:

```
git config --list
```

Step 5 — Check Repository Status

```
git status
```

Shows tracked, untracked, and staged files.

Step 6— Stage Files

git add <filename> (adds single file)

Or to add **all files**:

```
git add .
```

Step 7 — Commit Files

```
git commit -m "Initial commit"
```

Stores your changes in Git history.

Step 8 — View Commit History

```
git log
```

Displays the list of commits with IDs, authors, and messages.

Step 9 — Create a Remote Repository(Note: create the git hub account before proceeding)

- Go to https://github.com
- Click **New Repository**
- Name it **git-lab-project**
- Click **Create Repository**

Step 10 — Connect Local to Remote

git remote add origin <URL> //You need to copy the git hub repository URL

Links your local repo with the remote GitHub repo.

Step 11 — Push Code to GitHub

git push -u origin master

Uploads local commits to GitHub.

Step 12 — Clone Remote Repository

Create another folder on desktop with any name > open the folder > right click > click on open git bash here > and then give the below command

git clone <URL> // You need to copy the git hub repository URL

Creates a local copy of the remote repository.

Step 13 — Reset Changes

git reset

EXPERIMENT-2

Demonstrate to create a project in remote repository and apply fork, merge, diff, Merge Conflict, branch and pull request concepts on repository using GitHub.

NOTE: ALL THE COMMANDS YOU GIVE SHOULD BE IN SMALL LETTERS

Step 1:

Create a new repository in the git hub [This will be the remote repository]

Step 2: Clone repository to local machine

Create a folder on your desktop

Open git bash>give the command>

Git clone "URL"

Step 3: Create a file either manually

Or

Echo "Hello Github" > hello.txt

Add and commit:

Git add hello.txt

Git commit –m "added hello.txt file"

Git push origin main

Step 4: Create a new branch

git checkout –b feature-branch

Step 5: Edit the file

Echo "This is a feature branch edit" >> hello.txt

Then add, commit and push:

Git add hello.txt

Git commit –m “Updated hello.txt from feature-branch”

Git push origin feature-branch

Step 6: Create a pull request

1. Go to github repository
2. See merge “Compare and pull request”, Click
3. Add title and description
4. Create pull request

Step 7: Merge pull request

1. After review, click merge pull request
2. Click confirm message
3. Branch changes are now merged into master

Step 8: Demonstrate the merge conflict

1. Go back to master branch, give command

Git checkout master

2. Edit hello.txt

Echo ”Master branch edit” >> hello.txt

Git add hello.txt

Git commit –m “Edited in master branch”

3. Now switch to feature-branch again

Git checkout feature-branch

Edit the line:

Echo “Feature branch conflicting edit” >> hello.txt

Git add hello.txt

Git commit –m “conflicting edit in feature branch”

Try merging feature-branch into master branch

Git checkout master

Git merge feature-branch

//You will see a conflict message

Step 9: Resolve the conflict

Open hello.txt

// Remove all signs with <<<<,>>>,===

Then,

Git add hello.txt

Git commit –m “resolved merge conflict”

Git pull origin main

It opens the VIM later don’t type anything just press ESC and :wq and ENTER

Git push origin master

Step 10: View differences

Git diff //to compare file differences

Git diff master feature-branch // compare branch

Step 11: Fork the repository (For collab)

1. Go to the repository
2. Click fork

EXPERIMENT- 3

Demonstrate the process of integration github repository with Jenkins to automate the Project execution in CI/CD pipeline

1. Start Jenkins Service

Run Jenkins from the system

Open browser:

http://localhost:8080

Login to Jenkins Dashboard

2. Create GitHub Repository

2. Create a new repository in GitHub

Upload project using:

git init

git add .

git commit -m "Initial Code Upload"

git remote add origin <Repo URL>

git push -u origin main

3. Create New Project in Jenkins

Click New Item

Enter project name

Select Freestyle Project

Click OK

4. Integrate GitHub Repository

Go to Source Code Management

Choose Git

Paste repository link (HTTPS)

Add credentials if private repo

5. Add Build Step

Click Add Build Step

Example build command (for Maven):

mvn clean install

6. Save Configuration

Click Save

7. Manual Build Execution

Click Build Now

View build progress in Console Output

8. Verification

Modify code in GitHub and push

Again click Build Now to rerun CI build

EXPERIMENT-4

Create a Docker image for an application stored in a local repository and run the application using the Docker image.

1. Creating the Application Files

A basic Docker element is a **Dockerfile**, which contains step-by-step instructions for building a Docker image.

Before writing a Dockerfile, we first create an **HTML file** that will serve as our web application.

Create a new directory named **appdocker**, then create a file inside it using:

echo. > filename.txt

Create **index.html** and write the following content:

```
<html>
  <body>
    <h1>Welcome to my new app</h1>
    This page is test for my demo Dockerfile.<br />
    Enjoy ...
  </body>
</html>
```

2. Writing the Dockerfile

In the same directory (**appdocker**), create a file named **Dockerfile** (no extension) with the following content:

```
FROM httpd:latest
COPY index.html /usr/local/apache2/htdocs/
```

Explanation:

- **FROM httpd:latest** → uses Apache HTTP server as base image
- **COPY** → places our HTML file inside the Apache server directory

3. Building the Docker Image

Open a terminal inside the **appdocker** directory and execute:

docker build -t demobook:v1 .

Notes:

- Ensure Docker Desktop is installed and running.
- The **-t** option assigns the name (demobook) and tag (v1).
- The **.** indicates the current directory.

To verify the image is created, run:

docker images

4. Instantiating a New Container from the Image

Run the following command:

docker run -d --name demoapp -p 8080:80 demobook:v1

Explanation:

- **-d** → runs the container in background
- **--name demoapp** → container name
- **-p 8080:80** → maps local port 8080 to container port 80

To view running containers:

docker ps

5. Testing the Container Locally

Open a browser and enter:

http://localhost:8080

You should see the HTML page output:

"Welcome to my new app – This page is test for my demo Dockerfile. Enjoy..."

6. Pushing the Image to Docker Hub

Step 1: Login to Docker Hub:

docker login -u <your Docker Hub username>

Step 2: Retrieve the image ID:

docker images

Step 3: Tag the image for Docker Hub:

docker tag <imaged ID> <docker hub login>/demobook:v1

Step 4: Push the Docker image to Docker Hub:

docker push docker.io/<dockerhub login>/demobook:v1

EXPERIMENT-5

Create and configure Jenkins files for workflow and build of an application and push the image.

Step 1: Prepare Local Project Files

- Store index.html and Dockerfile inside Jenkins workspace folder.

Step 2: Create a Jenkins Pipeline Job

1. Open Jenkins → "New Item"
2. Select **Pipeline**
3. Provide name → Click **OK**

Step 3: Add Docker Hub Credentials

1. Go to **Manage Jenkins → Credentials**
2. Add new credentials

   Type: Username & Password

   ID: dockerhub_credentials

3. Save

Step 4: Configure Pipeline Script

Paste the following Jenkinsfile inside the job's pipeline section.

```
pipeline {
   agent any


   stages {
      stage('Build Docker Image') {
         steps {
            bat 'docker build -t nehapmca/demobook:v1 .'
         }
      }


      stage('Push Docker Image') {
```

```
        steps {
            withCredentials([usernamePassword(credentialsId: 'dockerhub_credentials', usernameVariable: 'USERNAME', passwordVariable: 'PASSWORD')]) {
                bat 'docker login -u %USERNAME% -p %PASSWORD%'
                bat 'docker push nehapmca/demobook:v1'
            }
        }
    }

    stage('Run Container') {
        steps {
            bat 'docker run -d -p 7070:80 nehapmca/demobook:v1'
        }
    }
  }
}
```

Step 5: Click "Build Now"

Go to your pipeline job:

Click **Build Now**

EXPERIMENT -6

Create a maven project with all dependencies required for the application in CI/CD pipeline.

1. Install and Configure Maven

Download Maven from the official website.
Set MAVEN_HOME environment variable.
Add Maven bin folder path to system PATH variable.

Verify using:

mvn -version

2. Create a Maven Project in IntelliJ IDEA

Go to: File → New → Project
Select Maven project
Provide project name and location
Finish to create default Maven directory structure:
src/main/java
src/test/java
src/main/resources
pom.xml

3. Configure pom.xml

Add Selenium and TestNG dependencies:

```
<dependency>
<groupId>org.seleniumhq.selenium</groupId>
<artifactId>selenium-java</artifactId>
<version>3.141.59</version>
</dependency>
<dependency>
<groupId>org.testng</groupId>
<artifactId>testng</artifactId>
<version>7.4.0</version>
<scope>test</scope>
</dependency>
```

4. Create Static Website Files

Under: src/main/resources

Create:
index.html
style.css
logo.png
Insert simple webpage content using HTML and CSS

5. Initialize Git Repository and Push to GitHub

In terminal:

git init
git add .
git commit -m "Initial Commit"
git remote add origin <GitHub-Repository-URL>
git push -u origin master

6. Configure Maven Resources Plugin

Add the following inside <build> tag in pom.xml:

```
<plugins>
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-resources-plugin</artifactId>
<version>3.2.0</version>
<executions>
<execution>
<phase>prepare-package</phase>
<goals><goal>copy-resources</goal></goals>
<configuration>
<outputDirectory>${project.basedir}/docs</outputDirectory>
<resources>
<resource>
<directory>src/main/resources</directory>
<includes>
<include>**/*</include>
</includes>
</resource>
</resources>
</configuration>
```

</execution>
</executions>
</plugin>
</plugins>

7. Build the Project

Run the command:

mvn clean install

Static website files will be copied to /docs folder

8. Deploy the Website using GitHub Pages

Push updated /docs folder:
git add docs/*
git commit -m "Deploy site"
git push

On GitHub:
Go to Settings → Pages
Choose:
Branch: main
Folder: /docs

Save and wait for publishing
Website gets hosted successfully

EXPERIMENT-7

Integrate communication channel with Jenkins for status of project and enable email notification for a build.

STEP 1: START JENKINS

1. Open a web browser
2. Enter the URL: **http://localhost:8080**
3. Login using Jenkins administrator credentials

STEP 2: VERIFY REQUIRED PLUGINS

1. Navigate to Manage Jenkins → Plugins
2. Ensure the following plugins are installed:
    - Git Plugin
    - Maven Integration Plugin
    - Email Extension Plugin
3. Restart Jenkins if any plugin is newly installed

STEP 3: CONFIGURE GLOBAL TOOLS

1. Go to Manage Jenkins → Global Tool Configuration
2. Configure the following tools:
    - JDK (Enable automatic installation or provide path)
    - Maven (Provide name such as Maven_3)
3. Save the configuration

STEP 4: CONFIGURE EMAIL NOTIFICATION IN JENKINS

1. Go to Manage Jenkins → Configure System
2. Scroll to Email Notification section
3. Enter the following details:
   SMTP Server: smtp.gmail.com
   Use SMTP Authentication: Yes
   Username: your_email_id
   Password: app_password
   SMTP Port: 465
   Use SSL: Yes
4. Set default email suffix
5. Test the email configuration
6. Save the settings

STEP 5: CREATE A JENKINS JOB

1. Click on New Item
2. Enter Job Name: Maven_Project_Build
3. Select Freestyle Project
4. Click OK

STEP 6: CONFIGURE SOURCE CODE MANAGEMENT

1. Select Git under Source Code Management
2. Enter the repository URL of the existing Maven project
3. Provide credentials if required
4. Select the branch (main or master)

STEP 7: CONFIGURE BUILD TRIGGER (OPTIONAL)

1. Select Build periodically or Poll SCM as required

STEP 8: CONFIGURE BUILD STEP

1. Click Add build step
2. Select Invoke top-level Maven targets
3. Configure Maven Version: Maven_3
4. Enter Goals:
   clean install

STEP 9: CONFIGURE EMAIL NOTIFICATION

1. Go to Post-build Actions
2. Click Add post-build action
3. Select Editable Email Notification
4. Configure the following:
   Project Recipient List: developer@email.com
   Triggers: Success, Failure, Unstable
   Subject: Build Status - ${BUILD_STATUS}
   Content:
   Project Name: ${PROJECT_NAME}
   Build Number: ${BUILD_NUMBER}
   Build Status: ${BUILD_STATUS}
   Build URL: ${BUILD_URL}
5. Save the job

STEP 10: BUILD THE PROJECT

1. Click Build Now
2. Observe the console output
3. Verify that an email notification is received

EXPERIMENT-8

Create a Maven project and integrate all dependencies, integrate the project with github repository, integrate with docker and docker hub and create Continuous Integration / Continuous Deployment pipeline using Jenkins and enable email notification for status of build.

STEP 1: CREATE MAVEN PROJECT

Open Command Prompt and execute:

mvn archetype:generate -DgroupId=com.example -DartifactId=demoapp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false

Navigate to project directory:

cd demoapp

STEP 2: ADD DEPENDENCIES (pom.xml)

Edit pom.xml and add dependencies (example):

<dependencies> <dependency> <groupId>junit</groupId> <artifactId>junit</artifactId> <version>4.13.2</version> <scope>test</scope> </dependency> </dependencies>

STEP 3: BUILD MAVEN PROJECT

Execute:

mvn clean install

Verify that BUILD SUCCESS is displayed.

STEP 4: INTEGRATE PROJECT WITH GITHUB

Create a repository in GitHub (example: demoapp)

Initialize Git:

git init
git add .
git commit -m "Initial Maven project commit"

Connect to GitHub:

```
git branch -M main
git remote add origin https://github.com/username/demoapp.git
git push -u origin main
```

STEP 5: CREATE DOCKERFILE

Create a file named Dockerfile inside the project directory:

```
FROM openjdk:11
WORKDIR /app
COPY target/demoapp-1.0-SNAPSHOT.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java","-jar","app.jar"]
```

STEP 6: BUILD DOCKER IMAGE

```
docker build -t username/demoapp:latest .
```

Verify image:

```
docker images
```

STEP 7: PUSH IMAGE TO DOCKER HUB

Login to Docker Hub:

```
docker login
```

Push image:

```
docker push username/demoapp:latest
```

STEP 8: CONFIGURE JENKINS TOOLS

Go to Manage Jenkins → Global Tool Configuration

Configure:
• JDK
• Maven
• Git

STEP 9: CONFIGURE EMAIL NOTIFICATION

Manage Jenkins → Configure System

Email Notification settings:

SMTP Server: smtp.gmail.com
SMTP Port: 465
Use SSL: Yes
Username: your_email@gmail.com
Password: app password

Test configuration and save.

STEP 10: CREATE JENKINS PIPELINE JOB

1. Click New Item
2. Enter Job Name: demoapp-ci-cd
3. Select Pipeline
4. Click OK

STEP 11: PIPELINE SCRIPT (Jenkinsfile)

```
pipeline {
agent any
tools {
maven 'Maven_3'
}
stages {
stage('Checkout') {
steps {
git 'https://github.com/username/demoapp.git'
}
}
stage('Build') {
steps {
sh 'mvn clean package'
}
}
stage('Docker Build') {
steps {
sh 'docker build -t username/demoapp:latest .'
}
}
```

```
stage('Docker Push') {
steps {
sh 'docker push username/demoapp:latest'
}
}
}
post {
success {
mail to: 'developer@email.com',
subject: 'Build Successful',
body: 'Jenkins build completed successfully.'
}
failure {
mail to: 'developer@email.com',
subject: 'Build Failed',
body: 'Jenkins build failed. Please check logs.'
}
}
}
```

STEP 12: RUN THE PIPELINE

Click Build Now
Observe pipeline stages
Check email notification